

# OSSP

## Practical Week -3

Roll No – 2520090085

Name - : Vakkalanka Sai Bhaskara Sujit

### Task -1:

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution

```
GNU nano 8.7.1 process_states.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

void print_process_state(const char *label, pid_t pid) {
    char path[64];
    char line[128];
    snprintf(path, sizeof(path), "/proc/%d/status", pid);
    FILE *fp = fopen(path, "r");
    if (!fp) {
        printf("[%s] PID: %d is Terminated / No longer exists.\n", label, pid);
        return;
    }
    printf("[%s] PID: %d, PPID: %d | ", label, pid, getppid());
    while (fgets(line, sizeof(line), fp)) {
        if (strncmp(line, "State:", 6) == 0) {
            line[strcspn(line, "\n")] = '\0';
            printf("%s\n", line);
            break;
        }
    }
    fclose(fp);
}

int main() {
    printf("=== Process State Demonstration ===\n");
    print_process_state("Initial Parent", getpid());

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) {
        // Child process
        printf("\n--- Child Process Started ---\n");
        print_process_state("Child Running", getpid());
    }
}
```

```

GNU nano 2.7.1 process_states.c
        break;
    }
}
fclose(fp);
}

int main() {
    printf("=== Process State Demonstration ===\n");
    print_process_state("Initial Parent", getpid());

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) {
        // Child process
        printf("\n--- Child Process Started ---\n");
        print_process_state("Child Running", getpid());

        printf("Child entering sleeping/waiting state (sleep 3s)...\n");
        sleep(3);

        print_process_state("Child Resumed", getpid());
        printf("--- Child Exiting ---\n\n");
        exit(0);
    }
    else {
        // Parent process
        sleep(1); // Give child time to enter sleep
        print_process_state("Parent Observing Child (Sleeping)", pid);

        // Wait for child to exit
        int status;
        wait(&status);

        printf("\n--- Child Terminated ---\n");
        print_process_state("Parent After wait()", getpid());
    }

    return 0;
}

```

Output:-

```

nocturnal@nocturnal:~/OSSP_2520090085$ nano state_demo.c
nocturnal@nocturnal:~/OSSP_2520090085$ gcc state_demo.c -o state_demo
./state_demo &
[2] 12262
State Demo PID: 12262
nocturnal@nocturnal:~/OSSP_2520090085$ ps -o pid,ppid,stat,cmd -C state_demo
  PID   PPID  STAT  CMD
  12262   322   S    ./state_demo
nocturnal@nocturnal:~/OSSP_2520090085$ cat /proc/$(pgrep state_demo)/status | grep -E "Name|State|Pid"
Name:   state_demo
State:  S (sleeping)
Pid:    12262
PPid:   322
TracerPid: 0
nocturnal@nocturnal:~/OSSP_2520090085$ kill -STOP $(pgrep state_demo)
ps -o pid,ppid,stat,cmd -C state_demo
  PID   PPID  STAT  CMD
  12262   322   T    ./state_demo

[2]+  Stopped                  ./state_demo
nocturnal@nocturnal:~/OSSP_2520090085$ kill -CONT $(pgrep state_demo)
ps -o pid,ppid,stat,cmd -C state_demo
  PID   PPID  STAT  CMD
  12262   322   S    ./state_demo
nocturnal@nocturnal:~/OSSP_2520090085$ kill -9 $(pgrep state_demo)
[2]-  Killed                    ./state_demo
nocturnal@nocturnal:~/OSSP_2520090085$ gcc process_states.c -o process_states
nocturnal@nocturnal:~/OSSP_2520090085$ kill %1

[1]+  Stopped                  nano process_states.c

[1]+  Stopped                  nano process_states.c
nocturnal@nocturnal:~/OSSP_2520090085$ [1]+  Terminated                  nano process_states.c
[1]+  command not found

```

## Task 2-:

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Input-:

```
GNU nano 2.7.1
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Process started\n");
    printf("PID : %d\n", getpid());
    printf("PPID : %d\n", getppid());

    printf("Process State: Running\n");

    printf("Process State: Sleeping/Waiting\n");
    sleep(10);

    printf("Process State: Running again\n");

    printf("Process State: Terminating\n");

    return 0;
}
```

Output-:

```
nocturnal@nocturnal:~/OSSP_2520090085$ ./task2 &
[2] 12569
Process started
PID : 12569
PPID : 322
Process State: Running
Process State: Sleeping/Waiting
nocturnal@nocturnal:~/OSSP_2520090085$ Process State: Running again
```

Using command Top():

```

> q
-bash: syntax error near unexpected token `q'
nocturnal@nocturnal:~/OSSP_2520090085$ top
top - 14:36:12 up 1:53, 1 user, load average: 0.02, 0.02, 0.00
Tasks: 27 total, 1 running, 25 sleeping, 1 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 11813.6 total, 10704.4 free, 643.7 used, 651.5 buff/cache
MiB Swap: 3072.0 total, 3072.0 free, 0.0 used, 11169.9 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
  1 root        20   0   24664   15832   11568 S   0.0   0.1   0:01.51 systemd
  2 root        20   0    3180    2208    2072 S   0.0   0.0   0:00.00 init-systemd(Ub
  6 root        20   0    3180    2044    1940 S   0.0   0.0   0:00.00 init
 43 root        19  -1   50424   16972   15652 S   0.0   0.1   0:00.23 systemd-journal
 78 systemd+    20   0   22540   14596   11976 S   0.0   0.1   0:00.05 systemd-resolve
 85 root        20   0   35456   12344    9156 S   0.0   0.1   0:01.06 systemd-udev
103 root        20   0    2888    1948    1820 S   0.0   0.0   0:00.01 chronyd-starter
104 root        20   0    4472    3136    2884 S   0.0   0.0   0:00.02 cron
105 message+    20   0    9188    5772    4688 S   0.0   0.0   0:00.28 dbus-daemon
157 root        20   0   44092   29800   14656 S   0.0   0.2   0:00.12 networkd-dispat
161 root        20   0   18612    9500    8200 S   0.0   0.1   0:00.10 systemd-logind
198 root        20   0    5492    2680    2500 S   0.0   0.0   0:00.00 agetty
207 _chrony      20   0   20420    6444    5668 S   0.0   0.1   0:00.10 chronyd
216 _chrony      20   0   12092    2424    1784 S   0.0   0.0   0:00.00 chronyd
228 root        20   0   123456   32556   17944 S   0.0   0.3   0:00.09 unattended-upgr
320 root        20   0    3184    1112     980 S   0.0   0.0   0:00.00 SessionLeader
321 root        20   0    3200    1248    1108 S   0.0   0.0   0:00.87 Relay(322)
322 nocturn+    20   0    6784    6164    3924 S   0.0   0.1   0:00.27 bash
323 root        20   0    8648    5532    4688 S   0.0   0.0   0:00.00 login
365 nocturn+    20   0   22684   13664   11032 S   0.0   0.1   0:00.12 systemd
367 nocturn+    20   0   22912    4112    2108 S   0.0   0.0   0:00.00 (sd-pam)
394 nocturn+    20   0    6136    5412    3860 S   0.0   0.0   0:00.01 bash
 844 polkitd     20   0   380176   8588    7708 S   0.0   0.1   0:00.26 polkitd
 8776 syslog      20   0   220548   5508    4664 S   0.0   0.0   0:00.03 rsyslogd
 9529 nocturn+    20   0    8544    5092    4672 S   0.0   0.0   0:00.00 dbus-daemon
12156 nocturn+   20   0    6036    5524    3124 T   0.0   0.0   0:00.04 nano
12593 nocturn+    20   0    8188    6020    3900 R   0.0   0.0   0:00.01 top

```

Using ps-ef command

```

root@nocturnal:~# ps-ef
ps-ef: command not found
root@nocturnal:~# ps -ef | grep week3pt2
root      12759   12703   0 14:43 pts/2    00:00:00 grep --color=auto week3pt2
root@nocturnal:~# |

```

**Observation:** The process was executed and monitored using Linux process-monitoring commands. The top command was used to observe the running process, while ps was used to identify the process and its PID. The experiment showed that a process can change between **Running** and Sleeping/Waiting states during execution. The process finally terminates after completing its execution.